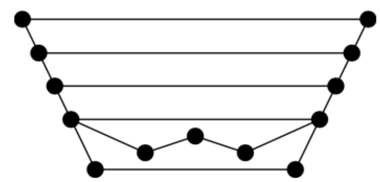
COT5520: COMPUTATIONAL GEOMETRY

Homework # 5

Due date: April 6, 2026, Monday

Your solutions should be concise, but complete, and typed (or handwritten clearly). Feel free to consult textbooks, journal and conference papers and also each other, but write the solutions yourself and cite your sources. **Answer only four (two among the first three and two among the last three) of the following six questions.** Each problem is worth 25 pts.

1. Draw the trapezoidal map $\mathcal{T}(S)$ and the directed acyclic graph of the search structure $\mathcal{D}$ for the set of segments S depicted here, for some insertion order of the segments.



2. Recall that in class we discussed a randomized algorithm to construct a trapezoidal map $\mathcal{T}(S)$ of a set of non-intersecting line segments S (together with a search structure $\mathcal{D}$) in $O(n \log n)$ expected time. Design a **deterministic** algorithm to compute the trapezoidal map $\mathcal{T}(S)$ of a set of non-crossing line segments S in $O(n \log n)$ time.
   [*Hint: Use the plane-sweep paradigm, and note that you are not required to compute* D.]

3. Given a polygon P and a query point q, we wish to determine whether q is inside P (without any preprocessing or building a special point location data structure).

   (a) Let P be a y-monotone polygon given as an array of its n vertices in counter-clockwise order along the boundary starting with the top-most vertex. Describe and analyze an algorithm that determines whether a query point q is inside P in $O(\log n)$ time.

   (b) Let P be a star shaped polygon given as an array of its n vertices in counter-clockwise order along the boundary. In addition, you are given a point p as a witness of the P being star shaped, that is, for any point r the segment pr lies in P. Describe and analyze an algorithm that determines whether a query point q is inside P in $O(\log n)$ time.

4. In Sections 10.1 and 10.2 we solved the problem of finding all horizontal line segments in a set that intersect a vertical segment. For this we used an interval tree with range trees or priority search trees as associated structures. There is also an alternative approach. We can use a 1-dimensional range tree on the y-coordinate of the segments to determine those segments whose y-coordinate lies in the y-range of the query segment. The resulting segments cannot lie above or below the query segment, but they may lie completely to the left or to the right of it. We get those segments in $O(\log n)$ canonical subsets. For each of these subsets we use as an associated structure an interval tree on the x-coordinates to find the segments that actually intersect the query segment.

   (a) Give the algorithm in pseudocode.

   (b) Prove that the data structure correctly solves the queries.

   (c) What are the bounds for preprocessing time, storage, and query time of this structure? Prove your answers.

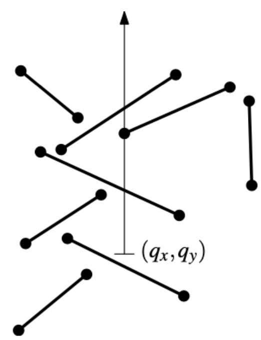
5. Let I be a set of intervals on the real line. We want to be able to count the number of intervals containing a query point in $O(\log n)$ time. Thus, the query time must be independent of the number of segments containing the query point.

   (a) Describe a data structure for this problem based on segment trees, which uses only $O(n)$ storage. Analyze the preprocessing time, and the query time of the data structure.

   (b) Describe a data structure for this problem based on interval trees. You should replace the lists associated with the nodes of the interval tree with other structures. Analyze the amount of storage, preprocessing time, and the query time of the data structure.

   (c) Describe a data structure for this problem based on a simple binary search tree. Your structure should have $O(n)$ storage and $(\log n)$ query time. (Hence, segment trees are actually not needed to solve this problem efficiently.)

6. We want to solve the following query problem: Given a set S of n disjoint line segments in the plane, determine those segments that intersect a vertical ray running from a point (q_x, q_y) vertically upwards to infinity.



   (a) Describe a data structure for this problem that uses $O(n \log n)$ storage and has a query time of $O(\log n + k)$, where k is the number of reported answers.

   (b) Now suppose we only want to report the first segment hit by the query ray. Describe a data structure for this problem with $O(n)$ expected storage and $O(\log n)$ expected query time. [Hint: Apply the locus approach.]